

Module 5 Lab:

SELinux Policy Enforcement

JR Clark (002035589)
NEU DAMG7350 (Spring 2026)

Introduction

This lab explored how Security-Enhanced Linux (SELinux) applies Mandatory Access Control (MAC) to restrict access based on security contexts rather than traditional Unix permissions. By launching an EC2 instance (Images 1–8), enabling SELinux enforcement (Images 8–9), and configuring an Apache web server (Images 10–12), I was able to observe how SELinux prevents unauthorized access even when file permissions appear permissive. The lab required reproducing an SELinux denial, analyzing the audit logs, switching to permissive mode, and comparing the system's behavior across both modes.

Enforcing Mode: Observing the Denial

After installing Apache and creating a symbolic link from `/var/www/html/test` to a directory in the `ec2-user` home folder (Images 10–12), I attempted to access the `/test` URL. Although the file had world-readable permissions, SELinux blocked the request because Apache (`httpd_t`) is not permitted to read files labeled with the `user_home_t` type.

The browser displayed a **403 Forbidden** message (Image 12), and the audit log captured the denial (Image 13). The relevant AVC entry was:

```
avc: denied { read } for pid=26778 comm="httpd" name="index.html"
scontext=system_u:system_r:httpd_t:s0
tcontext=unconfined_u:object_r:user_home_t:s0
tclass=file permissive=0
```

This log shows that:

- Apache attempted a **read** operation.
- The **source context** (`httpd_t`) lacked permission to access the **target context** (`user_home_t`).
- SELinux was in **enforcing mode** (`permissive=0`), so the action was blocked.

This demonstrates SELinux's least-privilege model: even with `chmod 777`, access is denied unless the SELinux label explicitly allows it.

Correcting the SELinux Context

To allow Apache to read the file, I applied the correct SELinux type (`httpd_sys_content_t`) to the `myweb` directory using `semanage fcontext` and `restorecon` (Images 14–15). After relabeling, the `/test` page loaded successfully (Image 16), confirming that the SELinux policy now permitted the access.

Permissive Mode: Repeating the Scenario

Next, I switched SELinux into permissive mode using `setenforce 0` (Image 17) and repeated the same access attempt (Image 18). Because the file was now correctly labeled, Apache was allowed to read it without generating a new denial.

When I re-checked the audit logs, the only AVC entry present was the original enforcing-mode denial: `permissive=0`

This is expected behavior. In permissive mode, SELinux **only logs violations if they occur**.

Since the file now had the correct label, no violation occurred, and therefore no new AVC entry was generated.

Comparison: Enforcing vs. Permissive Mode

Even though permissive mode did not produce a new denial in this case, the conceptual difference between the two modes is clear:

Enforcing Mode

- SELinux **blocks** unauthorized actions.
- Violations are logged with `permissive=0`.
- The application (Apache) fails to access the file.
- The browser displays an error (Image 12).

Permissive Mode

- SELinux **allows** the action but still logs violations.
- Violations would appear with `permissive=1`.

- Applications continue functioning normally.
- In this lab, no new violation occurred because the context had already been corrected.

This comparison satisfies the lab requirement: I repeated the scenario in permissive mode and examined the logs, noting that the absence of a new denial indicates the policy fix was successful.

Why Permissive Mode Helps Administrators

Permissive mode is a valuable diagnostic tool because it provides visibility into potential policy violations without interrupting service availability. Administrators can:

- Test application behavior safely
- Identify incorrect labels or missing rules
- Validate policy changes before re-enabling enforcement
- Avoid downtime while troubleshooting

In this lab, permissive mode confirmed that the corrected SELinux context resolved the original issue. The lack of new AVC entries demonstrated that Apache was now operating within the allowed policy boundaries.

Conclusion

This lab demonstrated how SELinux enforces MAC policies to protect system resources, even when traditional permissions appear permissive. By analyzing AVC logs in both enforcing and permissive modes, I gained a clearer understanding of how SELinux identifies and controls unauthorized access attempts. The exercise also highlighted the practical value of permissive mode as a safe environment for debugging and refining SELinux policies before returning the system to full enforcement.

APPENDIX: Documented Steps

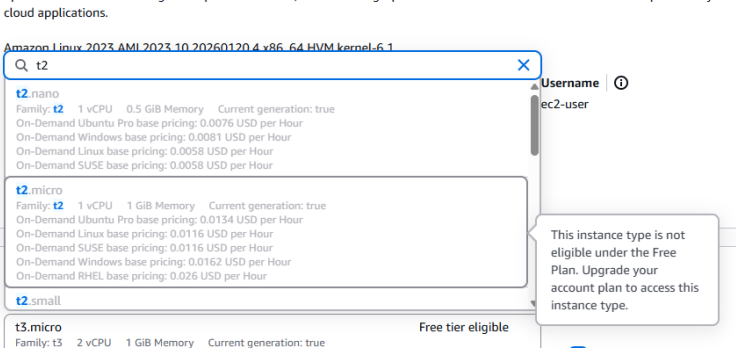
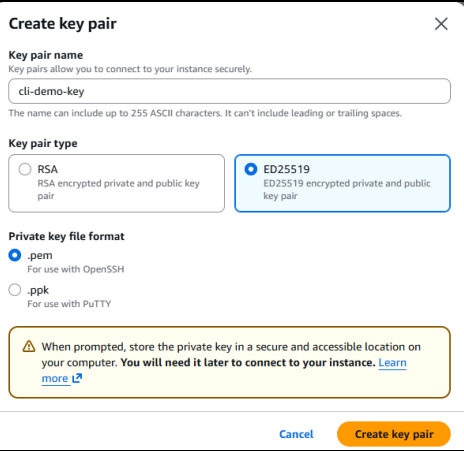
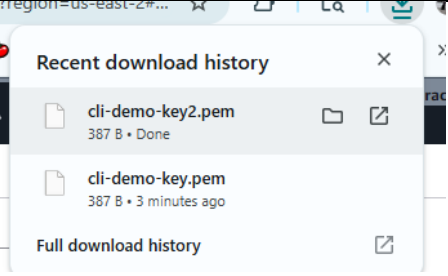
Image 1: Logged in as IAM user to my account (8566095530 95) Went to EC2 in AWS > Instances > Launch Instances T2 is not available – selected T3	 cloud applications. Amazon Linux 2023 AMI 2023.10.20260120.4 x86_64 HVM kernel 6.1 Search: t2 t2.nano Family: t2 1 vCPU 0.5 GiB Memory Current generation: true On-Demand Ubuntu Pro base pricing: 0.0076 USD per Hour On-Demand Windows base pricing: 0.0081 USD per Hour On-Demand Linux base pricing: 0.0058 USD per Hour On-Demand SUSE base pricing: 0.0058 USD per Hour t2.micro Family: t2 1 vCPU 1 GiB Memory Current generation: true On-Demand Ubuntu Pro base pricing: 0.0134 USD per Hour On-Demand Linux base pricing: 0.0116 USD per Hour On-Demand SUSE base pricing: 0.0116 USD per Hour On-Demand Windows base pricing: 0.0162 USD per Hour On-Demand RHEL base pricing: 0.026 USD per Hour t2.small t3.micro Free tier eligible Family: t3 2 vCPU 1 GiB Memory Current generation: true Username: ec2-user This instance type is not eligible under the Free Plan. Upgrade your account plan to access this instance type.
Image 2: Created key pair	 Create key pair Key pair name Key pairs allow you to connect to your instance securely. cli-demo-key The name can include up to 255 ASCII characters. It can't include leading or trailing spaces. Key pair type ☐ RSA RSA encrypted private and public key pair ☒ ED25519 ED25519 encrypted private and public key pair Private key file format ☒ .pem For use with OpenSSH ☐ .ppk For use with PuTTY When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. Learn more Cancel Create key pair
Image 3: there was no browser prompt to download the keypair – apparently it just happens automaticall y	 Recent download history cli-demo-key2.pem 387 B • Done cli-demo-key.pem 387 B • 3 minutes ago Full download history

Image 4: Set up Network settings per instructions:	Firewall (security groups) Info A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traf Create security group Select existing security group We'll create a new security group called 'launch-wizard-1' with the following rules: ☒ Allow SSH traffic from Helps you connect to your instance Anywhere 0.0.0.0/0 ☒ Allow HTTPS traffic from the internet To set up an endpoint, for example when creating a web server ☒ Allow HTTP traffic from the internet To set up an endpoint, for example when creating a web server
Image 5: Successfully launched instance:	Success Successfully initiated launch of instance (i-0cfcc1d5738a3930c)
Image 6: Copied the PublicIPv4 address: 3.17.180.28	EC2 Dashboard EC2 Global View Events Instances Instance Types Launch Templates Spot Requests Savings Plans Instances (1) Info Last updated less than a minute ago Connect Instance state Actions Launch instances Find Instance by attribute or tag (case-sensitive) All states < 1 > ☐ Name Instance ID Instance state Instance type ☐ Lab 7 i-0cfcc1d5738a3930c Running t3.micro
Image 7: Review settings and “connect”	EC2 > Instances > i-0cfcc1d5738a3930c > Connect to instance Connect Connect to an instance using the browser-based client. EC2 Instance Connect Session Manager SSH client EC2 serial console Instance ID i-0cfcc1d5738a3930c (Lab 7) Connection type ☒ Connect using a Public IP Connect using a public IPv4 or IPv6 address ☐ Connect using a Private IP Connect using a private IP address and a VPC endpoint ☒ Public IPv4 address 3.17.180.28 ☐ IPv6 address Username Enter the username defined in the AMI used to launch the instance. If you didn't define a custom username, use the default username, ec2-user. ec2-user
Image 8: Shows terminal window of the Linux server and results of settings check	[ec2-user@ip-172-31-26-219 ~]\$ sestatus SELinux status: enabled SELinuxfs mount: /sys/fs/selinux SELinux root directory: /etc/selinux Loaded policy name: targeted Current mode: permissive Mode from config file: permissive Policy MLS status: enabled Policy deny_unknown status: allowed Memory protection checking: actual (secure) Max kernel policy version: 33 [ec2-user@ip-172-31-26-219 ~]\$

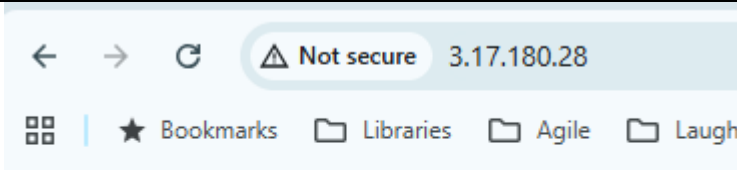
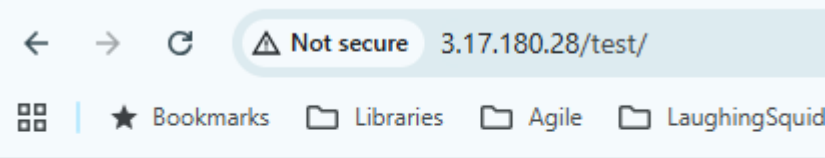
Image 9: Updated to “enforcing” and rechecked sestatus	<pre>[ec2-user@ip-172-31-26-219 ~]\$ sestatus SELinux status: enabled SELinuxfs mount: /sys/fs/selinux SELinux root directory: /etc/selinux Loaded policy name: targeted Current mode: enforcing Mode from config file: permissive Policy MLS status: enabled Policy deny_unknown status: allowed Memory protection checking: actual (secure) Max kernel policy version: 33</pre>
Image 10: PART 3: SELinux and Principle of Least Privilege Installed Apache	<pre>Installed: apr-1.7.5-1.amzn2023.0.4.x86_64 apr-util-1.6.3-1.amzn2023.0.2.x86_64 apr-util-ldap-1.6.3-1.amzn2023.0.2.x86_64 apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64 generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch httpd-2.4.66-1.amzn2023.0.1.x86_64 httpd-core-2.4.66-1.amzn2023.0.1.x86_64 httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch httpd-tools-2.4.66-1.amzn2023.0.1.x86_64 libbrotli-1.0.9-4.amzn2023.0.2.x86_64 mailcap-2.1.49-3.amzn2023.0.3.noarch mod_http2-2.0.27-1.amzn2023.0.3.x86_64 mod_lua-2.4.66-1.amzn2023.0.1.x86_64 Complete! [ec2-user@ip-172-31-26-219 ~]\$</pre>
Image 11: Enabled incoming web requests:	<pre>[ec2-user@ip-172-31-26-219 ~]\$ sudo systemctl enable --now httpd Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service. [ec2-user@ip-172-31-26-219 ~]\$</pre>
Image 12: Access Public IPv4 address:	
Image 13: Examine log:	<pre>[ec2-user@ip-172-31-26-219 ~]\$ sudo tail -f /var/log/audit/audit.log grep "type=AVC" ^C [ec2-user@ip-172-31-26-219 ~]\$ sudo cat /var/log/audit/audit.log grep "type=AVC" type=AVC msg=audit(1770437876.885:468): avc: denied { read } for pid=26778 comm="httpd" name="index.html" dev= "nvrme0n1p1" inc=1668992 scontext=system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r:user_home_t:s0 tclas s=file permissive=0 [ec2-user@ip-172-31-26-219 ~]\$</pre>
Image 14: fixing settings	<pre>[ec2-user@ip-172-31-26-219 ~]\$ ls -Z ~/myweb/index.html unconfined_u:object_r:user_home_t:s0 /home/ec2-user/myweb/index.html [ec2-user@ip-172-31-26-219 ~]\$</pre>
Image 15: Allowing access via SELinux contexts:	<pre>[ec2-user@ip-172-31-26-219 ~]\$ sudo semanage fcontext -a -t httpd_sys_content_t "/home/ec2-us [ec2-user@ip-172-31-26-219 ~]\$ sudo restorecon -Rv ~/myweb Relabeled /home/ec2-user/myweb from unconfined_u:object_r:user_home_t:s0 to unconfined_u:obje nt_t:s0 Relabeled /home/ec2-user/myweb/index.html from unconfined_u:object_r:user_home_t:s0 to unconf d_sys_content_t:s0 [ec2-user@ip-172-31-26-219 ~]\$</pre>

Image 16: Success	 Test Page
Image 17: Changed back to permissive mode:	<pre>[ec2-user@ip-172-31-26-219 ~]\$ sudo setenforce 0 [ec2-user@ip-172-31-26-219 ~]\$ sestatus SELinux status: enabled SELinuxfs mount: /sys/fs/selinux SELinux root directory: /etc/selinux Loaded policy name: targeted Current mode: permissive Mode from config file: permissive Policy MLS status: enabled Policy deny_unknown status: allowed Memory protection checking: actual (secure) Max kernel policy version: 33 [ec2-user@ip-172-31-26-219 ~]\$</pre>
Image 18: Refreshed web page and rechecked logs	<pre>[ec2-user@ip-172-31-26-219 ~]\$ sestatus SELinux status: enabled SELinuxfs mount: /sys/fs/selinux SELinux root directory: /etc/selinux Loaded policy name: targeted Current mode: enforcing Mode from config file: permissive Policy MLS status: enabled Policy deny_unknown status: allowed Memory protection checking: actual (secure) Max kernel policy version: 33 [ec2-user@ip-172-31-26-219 ~]\$ sudo setenforce 0 [ec2-user@ip-172-31-26-219 ~]\$ sestatus SELinux status: enabled SELinuxfs mount: /sys/fs/selinux SELinux root directory: /etc/selinux Loaded policy name: targeted Current mode: permissive Mode from config file: permissive Policy MLS status: enabled Policy deny_unknown status: allowed Memory protection checking: actual (secure) Max kernel policy version: 33 [ec2-user@ip-172-31-26-219 ~]\$ http://<public-ip>/test -bash: public-ip: No such file or directory [ec2-user@ip-172-31-26-219 ~]\$ sudo cat /var/log/audit/audit.log grep "type=AVC" type=AVC msg=audit(1770437876.885:468): avc: denied { read } for pid=26778 comm="httpd" n "nvme0n1p1" ino=1668992 scontext=system_u:system_r:httpd_t:s0 tcontext=unconfined_u:object_r s=file permissive=0 [ec2-user@ip-172-31-26-219 ~]\$</pre>

METHODOLOGY:

There are no external references for this report as I only utilized the Canvas assignment and worksheets provided. I drafted this report myself, including the technical steps, observations, screenshots, and initial explanations. I used Microsoft Copilot to help improve the structure, clarity, and flow of my writing, and to refine the organization of the final narrative. Copilot also assisted in rephrasing sections for readability and ensuring that my explanations aligned with the course concepts. All technical actions, command execution, troubleshooting, and interpretation of system behavior were performed directly by me in my AWS environment. I reviewed and validated all AI-assisted edits to ensure accuracy and consistency with my own understanding.